

Exercício final - Sistema Solar texturizado em OpenGL Moderno

Desenvolva, em **Python com OpenGL moderno**, uma simulação 3D do Sistema Solar contendo:

- Sol
- Mercúrio
- Vênus
- Terra
- Lua
- Marte
- Júpiter
- Saturno
- Urano
- Netuno

Cada corpo celeste deve ser representado por uma esfera com **textura própria**. As texturas podem ser baixadas no **NASA 3D Resources**, que reúne modelos, imagens e texturas disponibilizados pela NASA (<https://www.nasa.gov/3d-resources/>)

Também deve ser incluído um **skybox com textura de estrelas**. A textura do skybox pode ser obtida procurando por imagens/cubemaps livres de uso usando termos como:

- starfield cubemap free
- space skybox texture free
- stars cubemap OpenGL

Requisitos obrigatórios

A simulação deve representar o Sistema Solar em escala, usando os dados fornecidos abaixo. Use as seguintes escalas na aplicação:

```
distância_render = distância_real_em_milhões_de_km  
raio_render = raio_real_em_km / 50000
```

Para a Lua:

```
distância_render_da_Lua = 0.384
```

Ou seja, a Lua deve orbitar a Terra a uma distância proporcional à distância real média de aproximadamente **384000 km**.

Dados dos corpos celestes

Corpo	Diâmetro real km	Raio render	Distância ao centro da órbita milhões km	Período de rotação horas	Período orbital dias	Inclinação axial graus
Sol	1392700	13.927	0.0	609.1	0.0	7.25
Mercúrio	4879	0.049	57.9	1407.6	88.0	0.03
Vênus	12104	0.121	108.2	-5832.5	224.7	177.4
Terra	12756	0.128	149.6	23.9	365.2	23.5
Lua	3475	0.035	0.384 da Terra	655.7	27.3 ao redor da Terra	6.7
Marte	6792	0.068	228.0	24.6	687.0	25.2
Júpiter	142984	1.430	778.5	9.9	4331.0	3.1
Saturno	120536	1.205	1432.0	10.7	10747.0	26.7
Urano	51118	0.511	2867.0	-17.2	30589.0	97.8
Netuno	49528	0.495	4515.0	16.1	59800.0	28.3

Os dados de diâmetro, distância média ao Sol, período de rotação e período orbital seguem a tabela de fatos planetários da NASA; a Lua deve ser tratada como satélite da Terra, com rotação síncrona e órbita ao redor da Terra.

Movimento

Cada corpo deve ter dois movimentos:

1. Rotação em torno do próprio eixo

- Usar o período de rotação da tabela.
- Valores negativos indicam rotação retrógrada.

2. Translação orbital

- Os planetas orbitam o Sol.
- A Lua orbita a Terra.
- Usar o período orbital da tabela.

A velocidade pode ser acelerada por um fator global de simulação, por exemplo:

1 segundo real = 10 dias simulados

Mas as velocidades relativas entre os corpos devem respeitar os períodos reais.

Inclinação axial

Cada corpo deve ser renderizado com sua inclinação axial correta. A inclinação deve afetar o eixo de rotação do corpo, não a órbita.

Exemplo conceitual:

```
modelo = translação_orbital
        * rotação_orbital
        * inclinação_axial
        * rotação_do_próprio_eixo
        * escala_do_corpo
```

Iluminação

A cena deve usar iluminação **Blinn-Phong**. O **Sol deve ser a única fonte de luz** da cena.

Requisitos de iluminação:

- Luz pontual posicionada no centro do Sol
- Componente ambiente baixa
- Componente difusa
- Componente especular
- Cálculo com normal no fragment shader
- Cálculo do vetor de visão para o Blinn-Phong

O Sol não deve receber iluminação como os planetas. Ele deve ser renderizado como objeto emissivo.

Câmera e interação

A câmera deve permitir aproximar-se de qualquer corpo celeste usando as teclas:

- | | |
|-----------------|----------------|
| • 1 -> Sol | • 6 -> Marte |
| • 2 -> Mercúrio | • 7 -> Júpiter |
| • 3 -> Vênus | • 8 -> Saturno |
| • 4 -> Terra | • 9 -> Urano |
| • 5 -> Lua | • 0 -> Netuno |

Ao pressionar uma tecla, a câmera deve focar o corpo correspondente. O sistema também deve permitir navegação livre pela cena.

Entregáveis

O projeto deve conter:

- Código-fonte em Python
- Shaders GLSL usados na aplicação
- Texturas utilizadas
- Skybox de estrelas
- Arquivo **README .md** explicando:
 - como executar
 - quais bibliotecas foram usadas
 - quais teclas controlam a câmera
 - quais escalas foram adotadas
 - fonte das texturas utilizadas